COS Office Hours Queue: An Online Platform for Managing Office Hours

Kelly Petrino
Adviser: Dr. Robert Fish

Abstract

This paper outlines the design, development, and evaluation of an online platform designed to simplify COS office hours. The COS Office Hours Queue allows students to join a virtual queue to receive help during office hours. In addition to its surface functionality of improving the current queuing system of writing names on a crowded board, the COS Office Hours Queue matches students with similar questions together so that they can actively work on their problems with others while waiting for additional help. The application aims to make the office hours experience more simplistic and worthwhile for everyone involved and end the existing chaos that frequently ensues in Friend 010.

1. Introduction

1.1. Motivation

As a student concentrating in Computer Science (COS), I know all too well the feelings of confusion and defeat that arise when a student walks into an office hours session only to find it overrun with other students. This scene is practically synonymous with Friend 010, where most office hours sessions for the introductory level COS classes are held. Many times, there are more students with questions than can feasibly be helped under the current system where each student is attended to individually in the order of arrival to the classroom. In fact, in a survey of 10 Princeton students who have taken COS126 before about their experiences in office hours, 8 of them reported that they have had to leave office hours without getting any help because the wait was too long. From conversations with current course staff for COS126, I've gathered that the number of office hours

sessions has doubled this year to help ease some of the load, but that there are still times when the room is overrun with students. While surely the office hours system could solve this dilemma and benefit from large systematical changes like adding even more instructors and accessible spaces to hold additional sessions, this paper aims to find a more practical and easily implementable solution.

Adding to the confusion, often times, multiple classes will hold office hours in the same room at once, making the experience that much more chaotic. Currently, in order to receive help from an instructor, a student can join the queue by writing their name on the board. While this seems like the most simple system in theory, in practice, the whiteboards are overflowing with various different queues for various different classes. Sometimes, there are even old queues for the same classes up on the board, leading students to join a stale queue without even realizing it. This is clearly a solvable problem.

Beyond just the overall confusion surrounding how to get help, there is another fundamental flaw in office hours - time is being wasted due to repeat questions. Many students come in with the same, or very similar questions to ask. Rather than addressing these questions all at once in order to avoid repeating answers over and over, the current system forces course staff to help students one by one to maintain the structure of the queue. Students do not sit together, so it would be impossible for them to know who else is struggling with the same things. So, there needs to be a new system put in place to make students aware of each other and their common issues in order to group them together and avoid wasting time by asking repeat questions to the TA. This would allow the TA to assist more students with different questions and go through the queue faster.

1.2. Goal

For all these problems and more, office hours are a flawed system. We are clearly faced with a solvable issue - the office hours experience as it currently exists does not meet the needs of students or the course staff. Thus, my goal is to create a platform that allows students to join an online queue for COS office hours that makes the experience more efficient and simplistic for both students and course staff.

1.3. Overview

Given the problems outlined in the Motivation section, it is clear that there does not currently exist an efficient way for large groups of students to have their questions answered in COS Office Hours. In order to solve this issue, I propose developing a web application that has separate interfaces for both students and instructors, providing each party with the tools they need for a successful office hours session. On both ends, there needs to be an accessible queue that displays the students waiting for help and the expected wait time before receiving it. Additionally, on the student side, I want to design a matching program that links students with similar questions so that they can work together while they wait. As of right now, I want the app to be operational for COS126 office hours, but it has the potential to be expanded to other classes.

I implemented it using a 3-tier web application hosted on Heroku. The application's core (the back end) is a python server with Flask routing while the front end is mostly HTML/CSS with some Javascript and Jinja2. I also used a PostgreSQL database to store information about the queue and the operations of office hours (e.g. instructors, relevant key words to tag and describe typical questions). The website has a splash page where users can log in using CAS authentication and are then routed to the appropriate interface home based on their NetID. The student interface is home to the current queue with estimated wait time, a form that users can submit to join the queue, and, once they've joined, a panel that introduces them to their match. The instructor interface also displays the current queue and total wait time. In addition, it gives instructors the ability to remove any helped student, clear the queue at the end of a session, and add new instructors by inputting their NetID.

In order to test the application, I setup a simulated office hours session by gathering 10 former COS126 students to all use the application at once. Each test student was assigned a question and instructed to join the queue as though they had that question. The questions were pulled directly from the COS126 FAQ page for the final programming assignment and the Piazza page for the class. At the end of the simulated session, each student was asked to complete a survey evaluating the application's performance and utility. The results of this survey suggest that the online queuing system works as intended and can handle large surges of activity as it would during a real office

hours session. The results for the matching program were less positive, as 60% of students reported that they had some similarities with their match between their question, but that they probably could not actually work together. So, while the main goal of creating an online queue for office hours was successfully completed, the matching algorithm component surely has room for improvement.

In this paper, I will detail in full the research, development, and evaluation of the COS Office Hours Queue web application. Starting with the Problem Background, I will outline my sources of inspiration for the project and the research involved in design plans. Next, I will explain my approach for this project and how my design plan aims to solve the current problems we face in office hours. I will then detail the implementation of the web application, including changes to the original design made throughout development. Then, I will talk about the evaluation of the application, the results of that evaluation, and how that plan also changed over time. And finally, I will address the limitations I faced and outline the future work that can be done to improve the application and make it classroom ready for future COS126 classes.

2. Problem Background

2.1. Related Work

The inspiration for this project was the existing Lab TA Queue, which boasts an online queuing system for students to receive help debugging their assignments. Figure 1 shows the current queue, which broadcasts the names of students waiting for help and whether or not they are being helped. It also provides students with an estimated wait time. This wait time is based off of previous students time spent on the queue before having their status changed to "Being Helped" by a Lab TA. A notable exclusion from this public advertised list is the actual problems that students are facing. While this makes sense in the context of the project, the Lab TAs are meant to help with debugging issues, so collaboration would be problematic given the restrictions placed on intro course classes that prohibit students from looking at others code, in a more conceptual based environment, allowing students to work with each other could be incredibly beneficial to their own learning and also to the overall office hours experience.

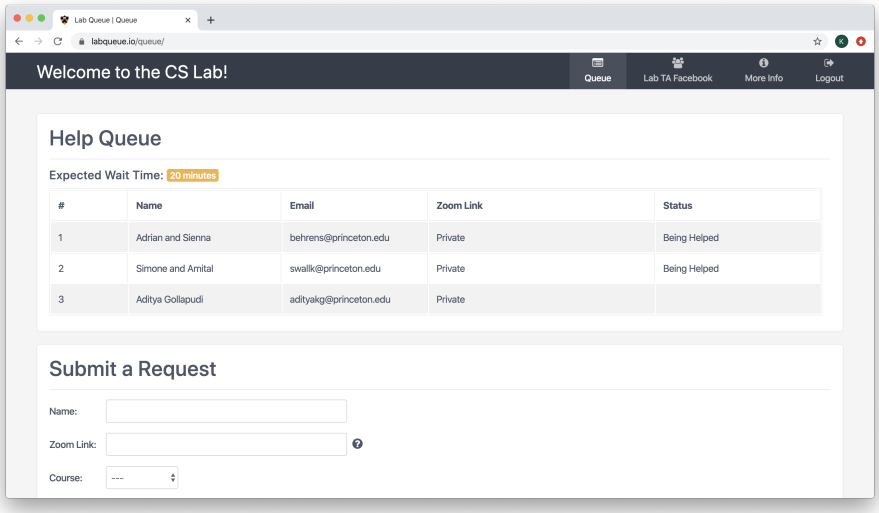

Figure 1: The queue for Lab TAs.

In order to join this queue, students fill out the form shown in Figure 2. The Lab TA Queue is built to handle the three introductory courses for the COS major, COS126, COS226, and COS217. You can choose which class you are currently enrolled in and briefly describe your problem in a free-form field when you join the queue. The basic design and implementation of the entire CS

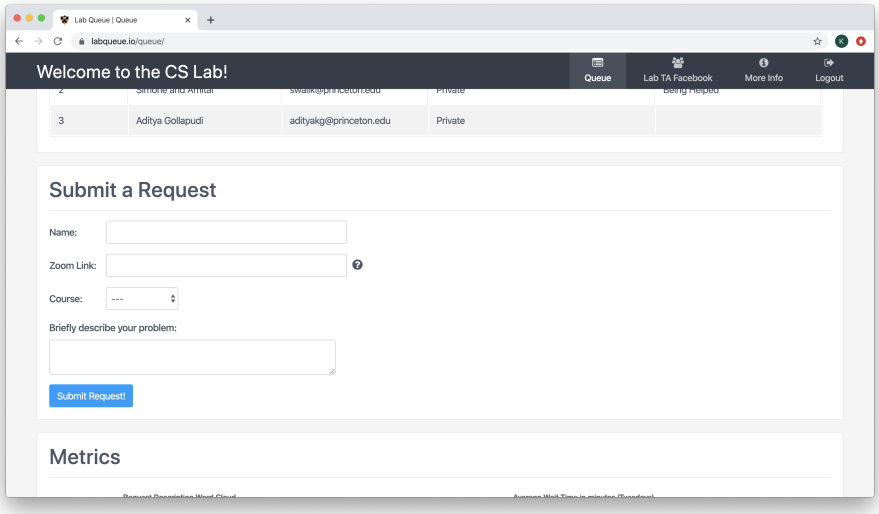

Figure 2: The form to join the queue for Lab TAs.

Lab website served as a baseline for my own design and implementation of the COS Office Hours Queue. In addition to serving as a inspiration for the project, the existing Lab TA Queue provided

me with valuable metrics information that prompted the idea for the novel component of my project, the matching program.

Shown in Figure 3, we can see that students are asking the same kinds of questions with their problem descriptions overwhelmingly consisting of the same keywords. From this screenshot taken

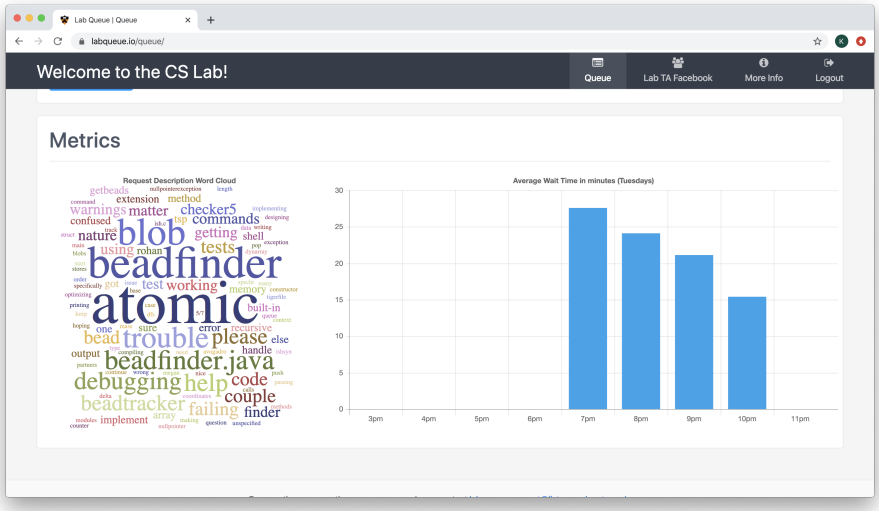

Figure 3: The metrics for Lab TAs.

a week before the final COS126 programming assignment (Atomic Nature of Matter) was due, we can see that some common questions are being asked. For example, the keyword "blob" means that students are struggling with the Blob data type that they need to implement for the assignment. If many students were struggling to understand what a Blob is, they could work together to talk about it conceptually. Another common keyword is "beadfinder.java," a class that implements a DFS. Students struggling to understand DFSs could also feasibly work together to understand the concept. These metrics reveal a critical shortcoming in the way we help students, we do not make it easy for them to work together. Thus, the COS Office Hours Queue was designed to match students with similar questions together to allow them to fill the dead space of waiting for a TA with active discussion and problem-solving. In fact, there is an abundance of academic research that supports this idea that students working collaboratively is beneficial to their learning.

2.2. Academic Background

Academics have been studying for years whether or not collaboration among students in beneficial or not. In just one simple Google search, dozens of articles will come up, every single one coming to the same conclusion, that collaboration is favorable to help students learn. When students collaborate, they are actively engaging in a discussion to further their knowledge. Princeton University hails itself for the one-on-one attention received by undergraduates and it's small class sizes. This is not for no reason. The university recognizes that by facilitating small group and individual discussions, students will be better equipped to retain the information they learn during their four years here. And even though there are some large lecture classes in which student participation is not involved, more often than not, there is also a precept to supplement it where participation is required and even incentivized via a participation grade. Thus, Princeton has created a culture in which students are encouraged to engage in conversations about the material. While Princeton has certainly been wrong on some things in the past, this collaboration culture is backed up by countless academics. In a study on small group collaborative learning (SGCL), researchers found that students who participated in SGCL showed an increase in individual critical thinking skills for science and math learning [3]. The same study also makes some claims about what they believe to be undergraduate best practices. The first three best practices, according to Quitadamo, are "productive faculty-student learning relationships," "student-student cooperation," and "collaborative discussion and peer critique" [3]. All of these facets, apparently the three most important things in an undergraduate program, support the notion of implementing a matching program in office hours that allows students with similar questions to discuss their problems together while they wait for instructor help. The first requirement is met via the close assistance of an instructor after actively trying to solve a problem, the second is met by the pairing of students to work together on the same problem, and the third is met by the discussion these students may have through this collaboration. For example, if two matched students don't have exactly the same problem, they could still help each other as the one further along could critique the other's progress and help them continue.

Peer collaboration teaches students to be persistent [2]. Studies show that students react positively to peer interaction and that it motivates them to actually learn the material [2]. This holds true in COS office hours. When students are left to their own devices, they are more likely to get distracted by other obligations and assignments while waiting for the instructor. By facilitating peer collaboration, students will be forced to be more engaged in the material they are there to receive help on as there is a social pressure to contribute to the discussion. There are also studies that suggest "collaboration may be a defining feature of competent and optimal practice, and the failure to collaborate may be indicative of negligence and malpractice" [1]. By not creating a way for students to collaborate, we would be doing a disservice to them as they are missing out on an opportunity to grow their own skills and be able to complete similar assignments on their own in the future which is an incredibly satisfying feeling.

For these reasons and so many more, it is clear that students should be given a platform to collaborate in office hours. Doing so would not only allow them to fill dead time with lively discussion and comprehension, but it will allow them to take on a more active role in their learning, developing their problem solving skills to be applied on future assignments and help them retain the knowledge they've gathered during their time at office hours rather than forgetting it immediately after as they would if they were to just scribble down whatever the instructor told them and promptly move on to something else.

3. Approach

3.1. Key Novel Idea - The Design Plan

With everything that's already been discussed in this paper, it is clear that there are some significant shortcomings in the way COS office hours are run. Simply, there does not currently exist an efficient way for large groups of students to have their questions answered in office hours.

In order to solve this problem I considered many different approaches. In every iteration of my design, the online queue served as the backbone. My original idea was to implement just this online queue for every COS class. I quickly realized, however, that the queue alone could not

accomplish the task of solving this problem. While the queue certainly makes queuing easier and more straightforward, the problems of scale would not be addressed and many students questions would continue to go unanswered. In talking to some of the current course staff of COS126 while I finalized the initial project plans, I was able pinpoint one of the most important issues with the current operation of office hours as the sheer number of students at peak times being too much to handle. Often times, instructors feel the need to go overtime in order to get to all the students. While there is surely some variability in the number of students that attend (for example, more students would be likely to attend right before an assignment is due or an exam is scheduled), it seems that a failure to address this problem would be a disservice to the course staff whose time and energy is incredibly valuable. The project also wouldn't be novel, simply a reiteration of the Lab TA online queue extended to office hours.

My solution to this problem aims to save time and answer a wider range of questions by eliminating repeat questions. I felt this would be an appropriate strategy because of my own experiences in COS classes and the experiences of others I surveyed. One of the hallmark phrases of a professor asking for questions is "Don't be afraid to ask, because I guarantee someone else is asking the same thing in their heads." I cannot count the number of times I've run into an issue while working on a COS assignment and upon checking Piazza, saw that someone else had already asked the same exact question and an answer was already posted. I considered a few options that never made it to fruition when deciding how to solve the problem of repeat questions in office hours in my application - using a similarity index to align students with the same questions together on the queue and combine them into one entry, giving students the ability to see others on the queues free form problem descriptions and add themselves to it manually, creating a matching program that automatically groups students based on the description of their problems so that they can find each other and work on the problem together.

Ultimately, I chose to implement the following approach which seemed most fit to fully address the needs of COS126 students and faculty. My design plan was to build a web application with different interfaces for both students and instructors, designed to address their unique needs. There

would be two key components to this application, the queue and the matching program. The current queue and the estimated wait time for students would be viewable at all times to all users of the site. On the student side, users should also be able to add themselves to queue and remove themselves in the case they need to leave office hours before they've been helped. When joining the queue, students should be able to provide a brief overview of what they are struggling with in order to give instructors some background before helping them and also to support the functionality of the other key component, the matching program. This matching program is the novel component of the project with its intent to be pairing students with the same question together to eliminate repeats and open up time in the session to help additional students. My design was to have each student be notified of their "match" once they join the queue. This match should be another student currently on the queue who described their issues in a similar way. Once notified, students can find their match in the room and sit with them to discuss their common issues, filling the previously dead space of waiting mindlessly. On the instructor side, the queue can be managed manually, allowing users to remove students on the queue as they are helped. It also should have the functionality to remove all of the students if an office hours session is over for that class and another is not immediately started to prevent stale queues from running over into later sessions. In the case where two students with the same question have been matched on their end and are helped together, the instructor can simply remove both students afterwards with that change reflected in the estimated wait time of the queue.

3.2. Why It Works

I believe that this approach completes the task of solving the problem established at the beginning of this section, that there does not exist a way for large groups of students to have their questions answered in office hours. Firstly, the online queue simplifies the system of queuing students at office hours. It has a clear start and end to the queue, unlike the current approach of writing names on a board, and makes it much easier to constantly manage it by removing students with just the click of a button. The inclusion of the estimated wait time also serves to help students make educated

decisions about whether or not they should join the queue given how much time they have to spend at office hours, in turn making the displayed queue more representative of the students who will actually be helped so that time will not be wasted looking for students who are no longer in the room. While the online queue alone has some effect in minimizing the dead time spent in office hours, freeing up space to help more students, it is really the matching program that is meant to make a noticeable impact. In theory, by matching students with similarities together, instructors should be able to address what would be repeat questions all at the same time. These students could find each other and, if their questions turn out to be the same, be helped simultaneously by the instructor such that two students can be removed from the queue at once. This will clearly free up time to help other students as the time that it would take help the students individually is halved.

Beyond that surface level effect, I also plan for the matching program to have a more long term effect on the time spent helping individual students on the queue by giving them an opportunity to take on a more active role in their learning. As addressed in the previous section (2), there is evidence to support the idea that collaborative learning is beneficial to students in the long run. Ideally, by matching students together they be able to discuss their issues at length with their partners before receiving help from the instructor. Instead of mindlessly regurgitating information given to them by instructors, students will be encouraged to actively solve their own problems, developing their own problem solving skills to be applied to future issues. In theory, this should decrease the amount of time spent helping individual students in the long run as they will have higher exposure to solving like problems and be more likely to understand an instructor's guidance faster. This decreased time spent helping individuals will free up time for instructors to reach more students in the allotted time slot for office hours.

4. Implementation

4.1. Problem Statement

As addressed in more detail in the Approach (Section 3), my design plan to solve the issue of there not being an efficient system to help large quantities of students in office hours involves developing

a web application that hosts an online queue and a matching program. In order to implement these features, I drew heavily upon my experiences in COS333 when I, as a part of a five-person team, helped to develop a web application with a python server and Flask routing. While that experienced prepared me for the development of a basic, functional web app, I also had to figure out how best to implement the queue and the matching program. In other words, what is the best way to store information about the students in the queue? How should I go about finding similarities in students? What kind of algorithm should I use to actually match them?

4.2. Application Overview

I implemented a 3-tier web application hosted on Heroku to implement my project plan. The front end uses HTML/CSS with Bootstrap, Javascript, and Jinja2. The back end is a Python server with Flask routing. Queue data and TA information is stored in a PostgreSQL database. When a client arrives on the website, they are met with a Splash Page (Figure 4) where they can login using their Princeton credentials. The web app uses CAS-authentication and redirects clients to the appropriate

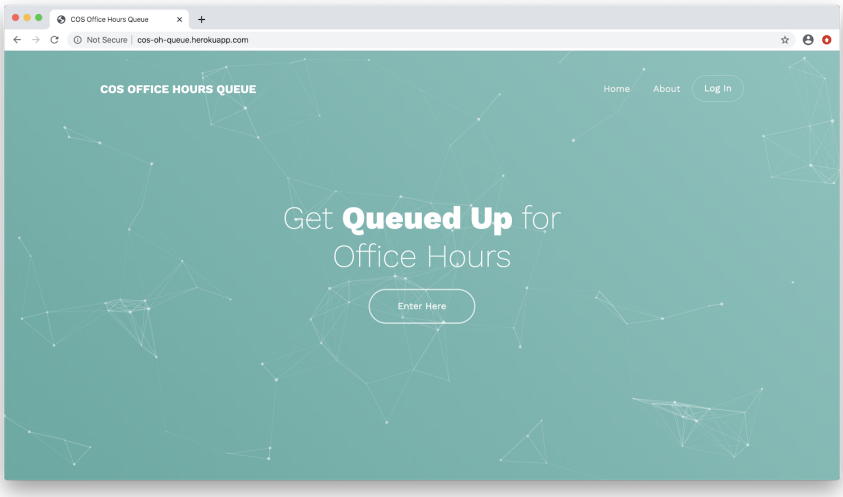

Figure 4: The Splash Page for COS Office Hours Queue.

home page based on their NetID. In other words, students will be directed to the student interface (Figure 5) while TAs are directed to the instructor interface (Figure 6). Students will be able to see the current queue, their estimated wait time, the form to join the queue, and, once they join, their

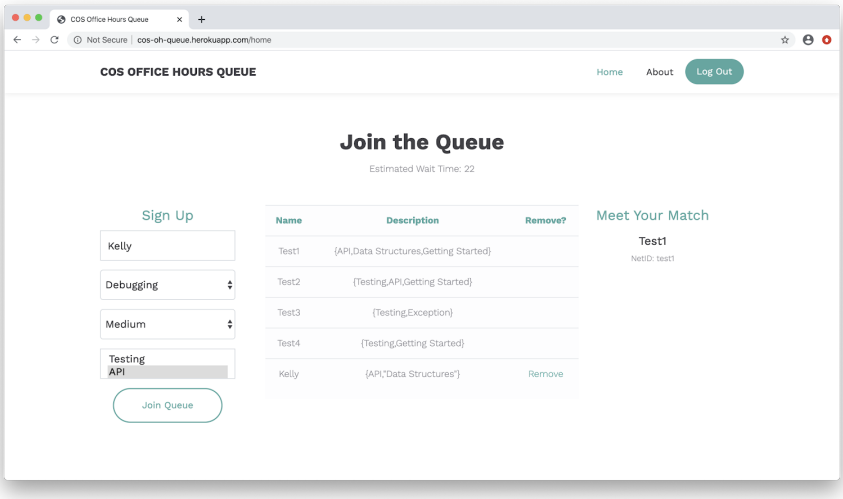

Figure 5: The student interface.

match. Once they've joined, students can remove themselves from the queue if they have to leave or have figured out their problem themselves. The form to join the queue asks students four simple questions - Name, Problem Type (Conceptual or Debugging), Problem Length (Quick, Average, or Long), and Problem Description. For the problem description field, students can choose as many options from a predetermined list as they feel necessary to describe their issues. This list is curated by the instructors and, ideally, updated according to relevant material for that week. The options are generally more broad in style, for example, API or Data Structures. The options which which it was evaluated are Testing, API, Data Structures, Algorithm, Exception, and Getting Started. Instructors

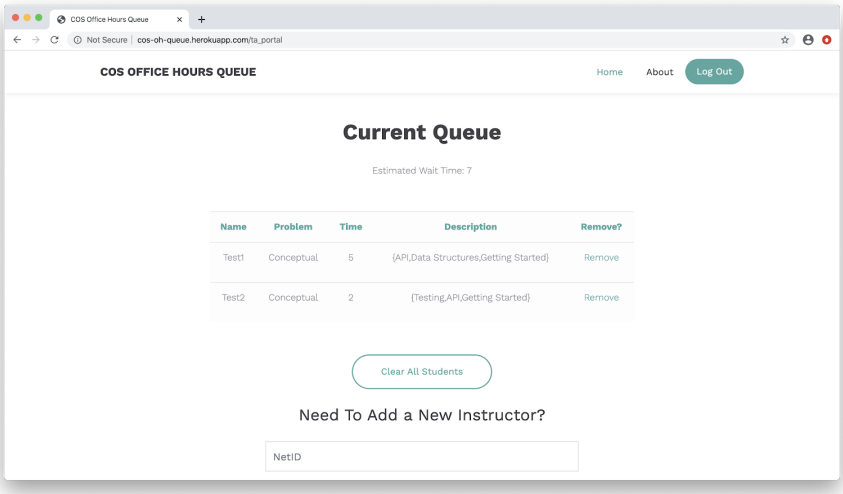

Figure 6: The instructor interface.

will be able to see the queue (with more details about student's problems), the estimated wait time to help all of the students, and form to add additional instructors. They can manage the queue by removing students as they are helped and clearing the entire queue at the end of the session.

4.3. Implementation Details

In implementing this web app, I relied heavily on my experience developing the web application CalcuLateMeal in COS333. That application also used a python server with Flask routing, preparing me to complete the baseline functionaly of the web app. My implementation timeline for project was approximately as follows:

- Weeks 1 and 2:
  - Develop a project idea
  - Get the project idea approved
- Week 3:
  - Create rough design plan for the application
- Week 4:
  - Survey current COS126 course staff
  - Get basic server hosting
  - Get approval for CAS Authentication
- Week 5:
  - Implement CAS Authentication
  - Implement basic form submission to join queue
- Week 6:
  - Add queuing system
  - Add wait time estimator
- Spring Break:
  - Add keyword selection to form submission
  - Add student matching program

- Week 7:
  - Develop instructor interface
  - Add dequeuing functionality
- Week 8:
  - Add limitation to one submission per NetID
  - Add remove self from queue functionality
- Week 9:
  - Add instructor clear all button
  - Revamp site UI
- Week 10:
  - Finalize site design and features
  - Deploy final design to Heroku
- Week 11:
  - Debugging
  - Run evaluation experiment
- Week 12:
  - Implement small changes from evaluation feedback
  - Analyze evaluation results

While there were some slight deviations from this schedule with tasks often running over into the following weeks, it is largely representative of the weekly effort put into the implementation of the COS Office Hours Queue. The timeline has been modified from my original project plan to reflect the features that made the final cut and the adjustments that had to be made given the limitations of virtual learning.

In terms of the server and routing mechanisms of the application, Flask made it incredibly easy to host a splash page, different interfaces, and send information through the back end via endpoints. The splash page endpoint is the first page users see. Upon clicking a button label "Enter Here," the user is redirected to the student interface endpoint which requires CAS login, denoted by the

tag `@login_required`. I used the library `Flask-CAS` to implement CAS functionality and read the authenticated ticket of each user. If the user is not already logged in, they will be redirected to the CAS login page and then back. Then, before rendering the page, the endpoint retrieves the NetID from the CAS ticket and if belongs to the set of instructors (stored in the database), then they are redirected to the instructor portal instead. (A side note to Dr. Fish: I have hardcoded you the same permissions I have to be able to use both interfaces. Upon login, you will be directed to the student interface, but you can access the instructor interface by manually going to the endpoint `/ta_portal`).

On the student side, the form is implemented using the library `Flask-WTF`. This library utilizes Jinja2 to streamline the sending of data submitted to a form on the front end into the back end. I defined a `SignUpForm` Flask form that had fields for the user to input their Name, Problem Type, Estimated Time, and Problem Description and then submit it. Then, on the back end, I can use the library to check if the form has been submitted using `<form>.is_submitted()`, and if it is, retrieve that values input and input them into a dictionary (to later be input into the PostgreSQL database) using `request.<form>.to_dict()`. To prevent multiple submissions, I have a check in place to see if the NetID submitting is already in the database before adding the new request. The same type of form is used on the TA interface to add additional instructors. After every request, the student interface is re-rendered with the updated queue and a match.

The matching program is based on a similarity index. The server has a helper method `jaccard` which takes in the problem descriptions of two students on the queue in the format of a `String` and returns the value of the similarity index computed. At a high level, a jaccard similarity index compares students based on their co-presences and their dissimilarities. Their co-absences are not taken into consideration. I chose this similarity index in lieu of others because I felt it was most suitable to the kinds of matches I was aiming to make. For example, if two students both chose not to select "API" as relevant to their problem, this does not indicate any actual similarities in their questions, just an absence of that singular topic which should not be a deciding factor in whether or not those students should work together. The algorithm for my matching program is as follows:

1. A new student joins the queue by submitting a form.
2. Upon submission, the back end reads in the data from this student and makes sure they are not already on the queue.
3. The back end iterates through the current queue, sending the problem description of the new student and an old student to the `jaccard` function. After each iteration, the program checks to see if the similarity index between the new student and that old student is greater than the previous maximum similarity index and updates the max accordingly.
4. The program outputs the student with which the new student has the highest similarity index.
5. The new student is added to the database and the page is re-rendered with the updated queue and the match.

The `jaccard` function is implemented by iterating through each possible key word that could have been input and using nested `if` statements that check if that key word is in the new student or old student's problem descriptions. Let a co-presence (cp) be when both students contain a same key word, a presence-absence (pa) be when the new student has a key word but the old one doesn't, and an absence-presence (ap) be when the new student doesn't have a key word but the old one does. The equation to compute the jaccard similarity index is $sim_{jaccard} = cp/(cp + pa + ap)$. While simple, this similarity index is efficient for our purposes of matching students who have similar problems at a high level.

The instructor interfaces main functionality is in managing the queue, so I had to figure out a way to easily remove students from the queue. I ultimately decided to include a POST endpoint that the user was directed to upon clicking the remove button rather than using another form. This is because I ran into problems implementing two forms on the same page. There are three endpoints for removing students - `/remove/<netid>` which takes as input the NetID of the single student to be removed by the instructor, `/remove_self/<netid>` which is used on the student interface for a user to remove themselves, and `/remove_all` which clears the entire table in the database with the current queue. Once the PostgreSQL command is run at these endpoints, the user is redirected to the endpoint from which they came.

The database is hosted on Heroku as well. I connected to it using the library `psycog2`. I also used this library to execute commands on the database, such as inputting new data to the queue or removing students form the queue. It has three tables - `queue` which stores the information about the current queue, `instructors` which stores the NetIDs of people with access to the instructor portal, and `problems` which stores the problem descriptions to be used in the multiple choice field on the form to join the queue.

The front end is developed from an open source template designed by ColorLib. It is composed of HTML, CSS, Javascript, and Bootstrap. I edited the HTML and CSS files to my liking for this project. The template in its original form can be found at https://colorlib.com/preview/#snipp. I used the Flask function `render_template()` to send relevant information to each render HTML file such as the content of the queue to be displayed, the estimated wait time, the student's NetID, and the students match from the back end to the front end.

The finished web application can be found at http://cos-oh-queue.herokuapp.com and the code base can be found at https://github.com/kellypetrino/cos-oh-queue.

5. Evaluation

5.1. Original Evaluation Plan

My initial plan to test the application was to go into actual COS126 office hours sessions and have them use the COS Office Hours Queue. My experiment setup was to observe office hours at two different times - in the week leading up to the first written exam and the week leading up to the second written exam. I chose exam weeks because they are times when students are more likely to ask conceptual questions rather than debugging questions, making it a more suitable environment for the matching program to be tested on. In the first week, I would just observe, collecting data about how long instructors spent with each student, how many students attended, and how long overtime the session ran in order to help all of the students. I would also listen in to the actual questions being asked to help serve as a basis for the problem description field I would later have to fill with options. In the second week, I would ask instructors to use my application, again collecting

data on how long students were being helped, the number of students, and how long after the session ran. The goal was to show that the application helped decrease the amount time spent with each student while increasing the total number of students being helped within the actual time constraints of the session. I would also solicit feedback from students after they had been helped in the form of a survey.

Of course, with the university going virtual for the second half of the semester, this plan had to scraped in favor of a Plan B that did not require me to actually go into the classroom and see the app put into use. Given the circumstances, it would have been nearly impossible to test the matching program component is a real virtual office hours session. Since all of the students and the instructor are all in the same Zoom room and cannot talk amongst themselves, collaboration amongst small subsets of students would have been hard to facilitate. Theoretically, students could join breakout rooms to discuss their problems and then wait for the instructor to then join that room, but it would give the instructor the additional task of monitoring and managing these breakout rooms, creating unnecessary additional work on their part. Thus, I instead chose to simulate an office hours session using a set of former COS126 students.

5.2. Experiment Design

In order to evaluate the application, I aimed to simulate an office hours session. I gathered 10 Princeton students, all of whom have taken COS126 before and attended office hours, to be the test students for this experiment while I served as the test TA. This simulation aimed to emulate an actual office hours session during the last week of classes when students would be working on their final programming assignment, Atomic Nature of Matter. I curated a list of example questions (Table 1) that could be used to join the queue. This list was drawn directly from actual questions asked on Piazza when I took COS126 and the frequently asked questions section of the Atomic Nature of Matter programming assignment. I pre-loaded the queue with 4 fake students so that all of the test subject would be matched with someone when they joined. The procedure for the experiment is as follows:

Participant	Question
Test1	What is the Blob data type?
Test2	How do I run BeadTracker?
Test3	BeadFinder not adding to existing Blobs.
Test4	What is the doubling hypothesis?
1	What is a reasonable runtime analysis for this assignment?
2	How do I search the images for Beads? Is there an algorithm I should use?
3	How do I know if my runtime is actually passing?
4	How should I name my variables?
5	I'm getting an IndexOutOfBounds Error in BeadTracker.
6	What should I be returning in BeadFinder?
7	Can I assume that the dimensions of all of the images are 640-by-480 pixels and that all of the runs consist of 200 frames?
8	I'm getting in error from StdOut in BeadTracker.
9	I'm getting a compiler error in BeadTracker.
10	I don't understand how to implement DFS

Table 1: Example Questions for Simluated Office Hours Session

1. Begin a Zoom session with the participants to monitor participation.
2. Randomly assign test questions to participants. (Table 1)
3. Have each test subject join the queue as if they have that question. Choices for the form fields are up to participants' discretions and interpretations of the questions.
4. Have test subjects discuss over Zoom with their matches whether or not their questions are compatible. (If paired with fake student, subject can evaluate ability to work together individually)
5. Remove subjects from queue periodically as if removing for being helped until all are removed.
6. Randomly selected 2 participants to test instructor functionality. Add them via the Instructor Portal.
7. Test subjects should join the queue again and test instructors should evaluate the instructor portal, removing students and clearing all at the end.
8. Have each test subject fill out a survey evaluating the performance of the application and to give feedback.

The survey was designed to gather feedback from users. It was split into three section - Previous Office Hours Experiences, Application Functionality, and Feedback. Overall, the application held up throughout the experiment, accomplishing its main goal of providing an effective online queuing

system for office hours. The experiment also gave valuable insight into the application's usefulness and changes to be made in both the short and long term to improve it.

5.3. Results

The most promising results from the evaluation were those pertaining to the functionality of the queue. The website did not crash when faced with the capacity of 10 students joining nearly at once as real students would at an office hours session. When surveyed, 80% of participants reported

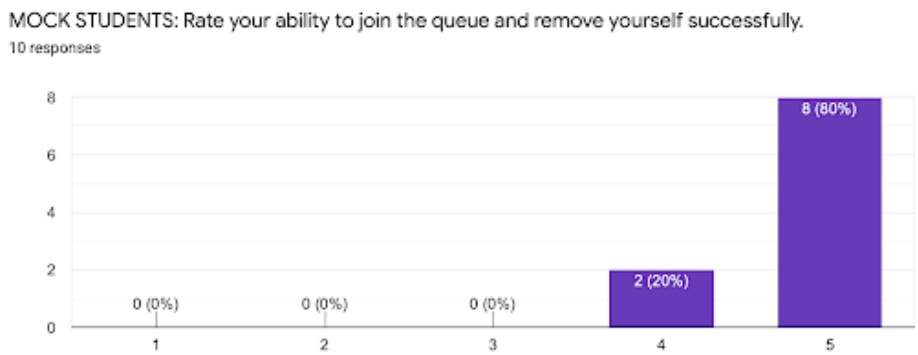

Figure 7: Results From Evaluation Survey, Queue Functionality

absolutely no problem joining the queue while 20% reported a very small problem. When asked to explain their rating, these students in the minority cited grievances with the forms fields and the options provided to them in it. Luckily, I also included a question to solicit feedback specifically on these forms, so I was able to pinpoint exactly which fields need work. From Figure 8, it is clear that the most flawed form field is that of Problem Description. As it currently stands, the field consists of a scrollable menu where students can select as many options from a pre-determined, generic list as they feel fit for their question. While many of the complaints surrounding this field had to do with its size (due to the template used, the size of the box is constricted to not show all the options at once, and it may not be intuitive that you can select muliple options), there was also a lot of feedback about the options themselves. Every participant reported that they believed the inclusion of a free form problem description field in addition to the choices would be beneficial. Many participants also reported that the choices themselves could've been more personalized to the

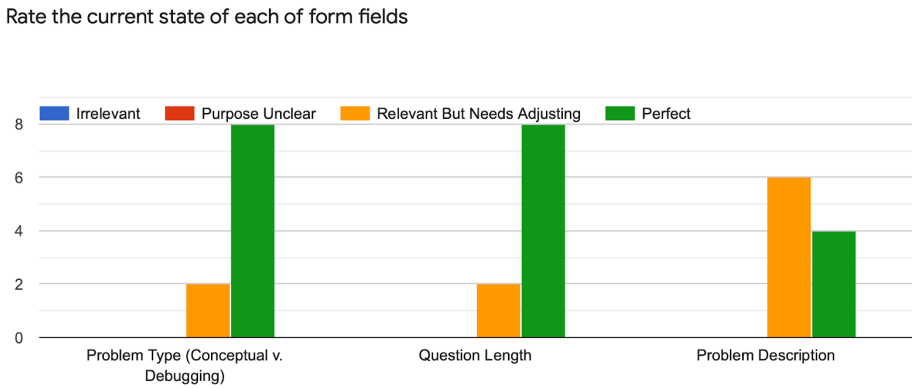

Figure 8: Results From Evaluation Survey, Form Fields

assignment. In other words, have the options be updated based on the assignment being dominantly asked about that week so that the matched questions might align more closely. Specifically for the final COS126 programming assignment, the options could have included things like "Blob," "BeadFinder," and "DFS" rather than the more generalized options like "Data Structures" and "API". This change would be likely to make the matches more realistic for working together.

Going off that feedback, it makes sense that the results pertaining to students' abilities to work on their problems with their matches were underwhelming. As seen in Figure 9, 70% of the

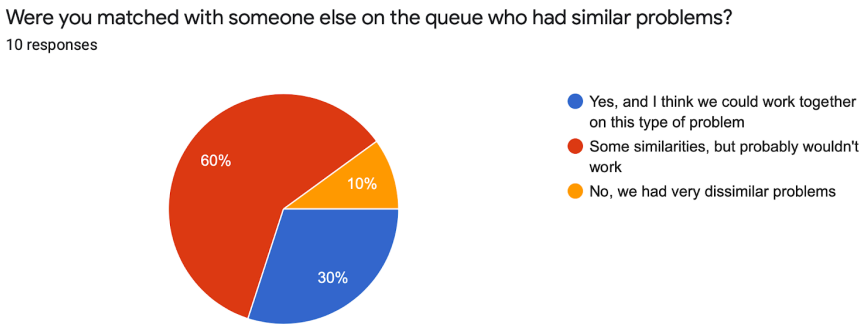

Figure 9: Results From Evaluation Survey, Matching Program

participants felt that the questions of their matches were not appropriate for collaboration with their own question. While it is clear that there are almost always some similarities between questions being matched, the participants did not believe that they could actually work with that person. There were multitude of justifications for these responses. Some cited concerns about the strict

collaboration policies in COS126 that prohibit students from looking at each others code. Others said that they're questions simply didn't have substantial enough overlap to foster a worthwhile discussion. For example, one participant who reported some similarities, but probably wouldn't work was assigned the question "How do I search the images for Beads? Is there an algorithm I should use?" and was matched with some with the question "What is the Blob data type?" While there is some overlap in that the Blob data type is what's used to store information about the Beads being searched for in images, it is unlikely that these two students would be able to work together as they seem to be at very different stages in the assignment. As mentioned before, changing the problem description field options could have prevented this match from happening and yielded a much better pairing for each of these students. However, an additional experiment would be required after implementing this change in order to test the validity of that claim. On the flip side, there were some students who were satisfied with their match and gave reassuring feedback about the program as a whole. For example, one student who reported that they felt they could definitely work with their match was assigned the question "I don't understand how to implement DFS" while their match has the question "How do I search the images for Beads? Is there an algorithm I should use?" As you can see, both of these questions pertain directly to the DFS algorithm used in BeadFinder.java. These two students could discuss how DFS works at a high level together and look at examples from precept to better understand how to implement it in their own programs. Then, an instructor could the overhead of explaining what DFS is when helping these students and get right to debugging their implementations if there were further issues. So, there is some promise when it comes to the matching program as when students were matched correctly, the feedback was overwhelmingly positive. In other words, the matching program is not inherently flawed in its use of a similarity index, but rather its results seem to have been skewed negatively by a bad choice of problem description options that were too general.

As it is currently implemented, the matching program certainly didn't perform as hoped, but the feedback and results give hope that with some small changes, it could have a much different outcome. By making the problem description fields more specific to the assignment that week and

additionally incorporating a free form problem description, the matching program would be more likely to provide students joining the queue with better matches and be able to live up to its goal of eliminating repeat questions and allowing students to take on a more active role in their learning experience via collaboration.

6. Summary

6.1. Conclusions

The COS Office Hour Queue serves as a means to make the experience at office hours more simplistic and accessible to students and instructors; however, it is only marginally successful in accomplishing its goal of making it possible to help a greater number of students in the same amount of time. In other words, the online queuing system works and is prepared to take on the sometimes massive onset of students attending any given office hours session, but the matching program, based on evaluation results, does not seem to be as useful as intended, especially in COS126. Given the nature of COS126 as an introductory class and the novelty that many students come into the course with, the results suggest a lack of faith in the ability of these students to be able to solve mildly similar questions by working together. Based on the feedback from evaluation participants, it seems the matching program could be more beneficial in upper level COS courses where collaboration is encouraged, specifically theory classes such as COS340 or COS445. Overall, despite the setbacks with the matching program, I feel that the COS Office Hours Queue could be a beneficial change implemented in the COS department.

6.2. Limitations

Obviously, I faced tremendous setbacks with the project given the onset of a global pandemic in the middle of the application's development. Beyond just delaying development by about a week and a half, the change in circumstances forced me to re-evaluate my approach to implementing the application and its evaluation. I talked more in detail about the shift to an Evaluation Plan B in that section (5). In terms of the change in design plan, I had to pick and choose the additional

features I chose to implement based on what I felt I could accomplish without in person guidance. For example, I would have liked to design a more complex matching algorithm, but felt that since I couldn't receive in person guidance about how to go about implementing it, it would be better to keep it simple.

Besides COVID-19, my largest limitation was my lack of experience on the front end. Because of this, I chose to use a free online template for the bulk of the design. However, because that template was so detailed and complex, I often struggled to understand how to make small changes to the design and ate up valuable time reading through documentation to do so. Should I do this project again, I would opt for a more simple template rather than go for the aesthetic factor.

6.3. Future Work

I believe that the COS Office Hours Queue has the potential to make a significant impact on the way COS office hours are run if it is further developed. Most importantly, I would like to expand the application to classes beyond just COS126. I could accomplish this by including a drop down on the splash page and having separate routes for each class based on what is selected. As mentioned before, I've concluded that the matching program would be much more applicable to some upper level COS classes so expanding the app to those classes should be a priority in future development.

In addition, I collected feedback from my evaluation participants about changes they would like to see made to the application. The most popular responses include updating the matching algorithm both to create groups larger than two and have matches be constantly updated as new students join, providing a box for free form problem description in addition to the drop down menu, and providing instructors with a feature to update the keywords being used for that problem description menu based on that week's assignment.

7. Acknowledgements

A big thank you to my advisor for this project, Dr. Robert Fish, along with my TA, Carlo Rosati, and the entire IW04 class for all of your input and guidance over the semester. I would also like to thank Grace Hong '21 for her help in choosing a project idea and my evaluation participants for giving valuable feedback about the functionality of the application and its future.

8. Ethics

This paper represents my own work in accordance with University regulations.

/Kelly Petrino/

References

[1] H. A. Lawson, "The logic of collaboration in education and the human services," *Journal of Interprofessional Care*, vol. 18, no. 3, pp. 225–237, 2004. Available: https://doi.org/10.1080/13561820410001731278

[2] B. Poellhuber, "The effect of peer collaboration and collaborative learning on self-efficacy and persistence in a learner-paced continuous intake model," *Journal of Distance Education*, vol. 22, no. 3, pp. 41–62, 2008. Available: https://eric.ed.gov/?id=EJ812561

[3] I. J. Quitadamo, "Critical thinking in higher education: The influence of teaching styles and peer collaboration on science and math learning," *ProQuest Disserations & Theses Global*, 2002. Available: https://search.proquest.com/docview/305497054?accountid=13314